```
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

def deutsch_jozsa(oracle):
    qc = QuantumCircuit(2,1)

    # Step 1: Initialize
    qc.x(1)  # output qubit = |1⟩

    # Step 2: Hadamard
    qc.h([0,1])

    # Step 3: Oracle
    qc.compose(oracle(), inplace=True)

    # Step 4: Hadamard on input
    qc.h(0)

    # Step 5: Measurement
    qc.measure(0,0)

    return qc
qc_const = deutsch_jozsa(constant_oracle)

sim = AerSimulator()
result = sim.run(qc_const, shots=1024).result()
counts_const = result.get_counts()

print("Constant Function:", counts_const)
plot_histogram(counts_const)
plt.show()
Constant Function: {'0': 1024}
qc_bal = deutsch_jozsa(balanced_oracle)

result = sim.run(qc_bal, shots=1024).result()
counts_bal = result.get_counts()

print("Balanced Function:", counts_bal)
plot_histogram(counts_bal)
plt.show()
Balanced Function: {'1': 1024}
```

1. What problem are you solving?

This code implements the **Deutsch–Jozsa Algorithm**.

Goal (very simple):

You have a function $f(x)$. It is guaranteed to be either:

- **Constant** → same output for all inputs
- **Balanced** → half inputs → 0, half → 1

👉 Task: **Find which one using minimum queries**

---

2. Classical vs Quantum Insight

Approach	Queries needed
Classical	Up to $2^{n-1} + 1$
Quantum (Deutsch–Jozsa)	**1**

👉 That's the power of quantum parallelism.

---

3. Your Code — Step-by-Step (Simple)

🔹 Oracle Functions

Constant Oracle

```python
def constant_oracle
    qc = QuantumCircuit 2
    return qc
```

👉 Does nothing
👉 Output always 0

✔️ So function = constant

Balanced Oracle

```python
qc.   0 1
```

👉 If input = 1 → flip output

Input	Output
0	0
1	1

✔️ Half 0, half 1 → balanced

---

4. Main Circuit

```python
qc = QuantumCircuit 2 1
```

👉 2 qubits:

- q0 → input
- q1 → output

Step 1: Initialize

```python
qc.  1
```

👉 Output qubit = |1⟩

Step 2: Hadamard

```python
qc.   0 1
```

Now:

Input qubit:

$$|0\rangle \to \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Output qubit:

$$|1\rangle \to \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

👉 Important:

- Output qubit becomes **phase carrier**

Step 3: Oracle

```python
qc.        oracle    inplace=True
```

👉 Oracle encodes function into **phase**, not value

Key idea (VERY IMPORTANT)

$|x\rangle \to (-1)^{f(x)}|x\rangle$

👉 Meaning:

- If f(x)=0 → no change
- If f(x)=1 → sign flip

Step 4: Hadamard on input

```python
qc.  0
```

👉 Converts phase → measurable result

Step 5: Measurement

```python
qc.        0 0
```

👉 Now you read result

---

5. Final Results (Your Output)

Constant Function

```python
'0'  1024
```

👉 Always 0

✔️ Means function is constant

Balanced Function

```python
'1'  1024
```

👉 Always 1

✔️ Means function is balanced

---

6. Why this works (core intuition)

Classical thinking:

You check outputs one by one

Quantum trick:

👉 Check **all inputs at once**

Because:

- Superposition → evaluates all inputs
- Phase → stores result
- Interference → extracts answer

---

7. Deep Insight (important)

👉 Algorithm does NOT compute f(x) directly
👉 It detects **pattern of f(x)**

8. Common Mistakes (you should avoid)

Mistake 1:

> "Oracle gives output directly"

❌ Wrong
✔ It modifies **phase**, not output

Mistake 2:

> "Why output qubit not measured?"

✔ Because:

- It is only used for **phase kickback**

Mistake 3:

> "Why always perfect result?"

✔ Because:

- No noise in simulator
- Ideal quantum interference

---

9. Debugging Workflow

If result is wrong:

Check 1:

- Did you apply `X(1)` before Hadamard?

👉 If not → algorithm fails

Check 2:

- Oracle correct?

👉 Wrong mapping → wrong classification

Check 3:

- Final Hadamard applied?

👉 Without it → no interference

---

10. One-Line Intuition

👉 Deutsch–Jozsa =
"Use interference to detect if function is constant or balanced in one shot"

---

11. Viva-Ready Summary

- Uses superposition + interference
- Oracle encodes function in phase
- One measurement decides answer
- Demonstrates quantum speedup

If you want next level

I can:

- ↳ derive full amplitude math step-by-step
- ↳ extend to n-qubit Deutsch–Jozsa
- ↳ compare with classical complexity formally

Just specify.